



# Personalized Networking Assistant

*Project Engineering Report & Deployment Lifecycle Manifest*

**Project Team:** Vyshnavi Mutyalapalli, Rahul Dev Bontha, Bommanaboina Nagaraju, Nishitha Kodi

**Architecture:** FastAPI + Streamlit + SQLite

## 1. Executive Summary & Architecture Overview

This document presents the complete engineering review of the **Personalized Networking Assistant**. The application features context-aware processing of corporate event briefs to generate dynamic conversation prompts, integrates live semantic documentation lookups via external web APIs, and utilizes structured state tracking to permanently retain user auditing logs and evaluation matrices.

The system is built entirely on a modular **3-Tier Architecture**:

- **Presentation Tier (Frontend):** Designed using Streamlit to offer an intuitive user panel divided into tabbed navigation screens ("Generate Starters", "Fact Verification", and "History & Feedback").
- **Logic Tier (Backend):** Programmed with FastAPI to handle background execution parameters and communicate with structural business services through asynchronous REST routes.
- **Data Tier (Persistence):** Powered by an integrated SQLAlchemy Object-Relational Mapper (ORM) linked to a standalone SQLite micro-database engine (`networking\_assistant.db`) for permanent storage.

## 2. Complete Module Status Matrix (Sprint Backlog Ledger)

The matrix below tracks all system task assignments mapped across team members, newly incorporating core deployment ownership, demonstrating 100% functional completion:

| Task / Module Name                           | Epic Reference                           | Assigned Engineer     | Status    |
|----------------------------------------------|------------------------------------------|-----------------------|-----------|
| <b>Description of the ER Diagram</b>         | Epic: Entity-Relationship Diagram        | Bommanaboina Nagaraju | Completed |
| <b>Project Workflow &amp; Pre-requisites</b> | Epic: Project Flow                       | Bommanaboina Nagaraju | Completed |
| <b>Model Research &amp; Selection</b>        | Epic 1: Model Selection and Architecture | Rahul Dev Bontha      | Completed |
| <b>Defining the Application Architecture</b> | Epic 1: Model Selection and Architecture | Rahul Dev Bontha      | Completed |
| <b>Environment Setup</b>                     | Epic 1: Model Selection and Architecture | Vyshnavi Mutyalapalli | Completed |
| <b>Data Schema Definition</b>                | Epic 2: Core Functionalities Development | Rahul Dev Bontha      | Completed |
| <b>Event Analyzer Service</b>                | Epic 2: Core Functionalities Development | Vyshnavi Mutyalapalli | Completed |
| <b>Topic Generator Service</b>               | Epic 2: Core Functionalities Development | Vyshnavi Mutyalapalli | Completed |
| <b>Fact Checker Service</b>                  | Epic 2: Core Functionalities Development | Vyshnavi Mutyalapalli | Completed |
| <b>History Logger Service</b>                | Epic 2: Core Functionalities Development | Bommanaboina Nagaraju | Completed |
| <b>Feedback Logger Service</b>               | Epic 2: Core Functionalities Development | Vyshnavi Mutyalapalli | Completed |
| <b>API Routes Design</b>                     | Epic 2: Core Functionalities Development | Rahul Dev Bontha      | Completed |
| <b>Application Entry Point</b>               | Epic 2: Core Functionalities Development | Vyshnavi Mutyalapalli | Completed |
| <b>The Centralized Routing Architecture</b>  | Epic 3: Main Application Logic           | Rahul Dev Bontha      | Completed |
| <b>Service Layer Design Principles</b>       | Epic 3: Main Application Logic           | Bommanaboina Nagaraju | Completed |

| Task / Module Name                                  | Epic Reference                             | Assigned Engineer     | Status    |
|-----------------------------------------------------|--------------------------------------------|-----------------------|-----------|
| <b>FastAPI-Specific Features Leveraged</b>          | Epic 3: Main Application Logic             | Rahul Dev Bontha      | Completed |
| <b>Application Setup and Configuration</b>          | Epic 4: Frontend UI Using Streamlit        | Bommanaboina Nagaraju | Completed |
| <b>Input Section and Main Generation Flow</b>       | Epic 4: Frontend UI Using Streamlit        | Rahul Dev Bontha      | Completed |
| <b>Results Display and Feedback System</b>          | Epic 4: Frontend UI Using Streamlit        | Bommanaboina Nagaraju | Completed |
| <b>Fact-Checking Section</b>                        | Epic 4: Frontend UI Using Streamlit        | Vyshnavi Mutyalapalli | Completed |
| <b>Conversation History View</b>                    | Epic 4: Frontend UI Using Streamlit        | Rahul Dev Bontha      | Completed |
| <b>Feedback History View</b>                        | Epic 4: Frontend UI Using Streamlit        | Vyshnavi Mutyalapalli | Completed |
| <b>Streamlit State Management Philosophy</b>        | Epic 4: Frontend UI Using Streamlit        | Rahul Dev Bontha      | Completed |
| <b>Testing Philosophy &amp; Framework Selection</b> | Epic 5: Testing and Local Deployment       | Bommanaboina Nagaraju | Completed |
| <b>Testing the Service Layers</b>                   | Epic 5: Testing and Local Deployment       | Rahul Dev Bontha      | Completed |
| <b>Testing API Routes with httpx</b>                | Epic 5: Testing and Local Deployment       | Vyshnavi Mutyalapalli | Completed |
| <b>Dockerization and Container Setup</b>            | Epic 6: Production Deployment Architecture | Nishitha Kodi         | Completed |
| <b>Cloud Deployment Orchestration &amp; Run Ops</b> | Epic 6: Production Deployment Architecture | Nishitha Kodi         | Completed |
| <b>Manual Production Testing &amp; Verification</b> | Epic 6: Production Deployment Architecture | Nishitha Kodi         | Completed |

### 3. Production Deployment Process

To transition the application from a local environment to a robust production infrastructure, a containerized deployment strategy using Docker is implemented, ensuring environmental parity and clean isolation.

#### Step 1: Containerization with Docker

Separate production Dockerfiles are structured for both the frontend and backend service configurations. The entire orchestration is consolidated into a root `docker-compose.yml` manifest:

```
version: '3.8'

services:
  backend:
    build: ./backend
    container_name: networking_backend
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app
    environment:
      - DATABASE_URL=sqlite:///./networking_assistant.db
    restart: always

  frontend:
    build: ./frontend
    container_name: networking_frontend
    ports:
      - "8501:8501"
    depends_on:
      - backend
    restart: always
```

#### Step 2: Launching the Microservices Stack

Run the following command from the project root directory to automatically build images, link networks, stand up containers in detached background mode, and initialize the SQLite storage database layer:

```
docker-compose up --build -d
```

#### Step 3: Verification Logs and Network Mapping

- **Backend Live Health-Check:** Accessible via production mapping at `http://<YOUR_SERVER_IP>:8000/docs`.

- **Frontend Application:** Served continuously to user channels at `http://<YOUR_SERVER_IP>:8501` .

## 4. Database Entity-Relationship (ER) Definition

Data schema records map configurations natively into a central relational mapping module managed within `models.py` :

Table: conversation\_history

| Field Name  | Data Type | Constraints                         |
|-------------|-----------|-------------------------------------|
| id          | INTEGER   | Primary Key, Auto-Incremented       |
| description | VARCHAR   | Index Tracked Text                  |
| themes      | VARCHAR   | Comma-separated array format values |
| starters    | VARCHAR   | Pipe-separated string prompt blocks |
| useful      | BOOLEAN   | Nullable evaluation indicator value |

## 5. Validation Framework & Conclusion

The microservices architectures have successfully undergone containerized regression testing and continuous integration checks. Database instances successfully maintain transaction records across the multi-container isolated setup, signaling definitive production deployment readiness for global platform delivery.